

Formal Verification: Wave Threshold Voting Engine

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize the Wave protocol, the core threshold voting engine of Lux consensus. Each round samples k peers, compares votes against an α threshold, and accumulates confidence toward the β finalization target. We prove that decided items are stable, confidence is monotone on matching rounds, and finalization occurs after β consecutive confirming rounds.

1 Introduction

Wave is the threshold voting engine at the core of Lux consensus. It implements the sampling-based voting loop described in `protocol/wave/wave.go`. The FPC variant selects $\alpha = \lceil \theta \cdot k \rceil$ where θ is PRF-selected per phase, providing additional Byzantine resilience.

2 Definitions

Definition 1 (WaveConfig). *Configuration: k (sample size), α (threshold), β (confidence target), with $0 < k$, $\alpha \leq k$, $0 < \beta$.*

Definition 2 (WaveState). *Per-item state: decided flag, preference (commit/skip), confidence counter.*

Definition 3 (RoundResult). *A round outcome: yes votes and total votes.*

3 Theorems and Axioms

Theorem 4 (`decided_is_stable`). *Once decided, the step function returns the same state: $\text{step}(\text{cfg}, s, r) = s$ when $s.\text{decided} = \text{true}$.*

Proof. By `simp` on the early-return branch. □

Theorem 5 (`confidence_monotone_on_match`). *When preference matches threshold ($\text{yesVotes} \geq \alpha$ and preference is true), confidence increments by 1.*

Proof. By case analysis on the step function branches. □

Theorem 6 (`finalization_after_beta`). *When confidence $+1 \geq \beta$ and the threshold is met with matching preference, the item is decided.*

Proof. By `simp` on the step function and `omega`. □

4 Lean Source

```
/-
Wave Protocol Formal Model

Models protocol/wave/wave.go and protocol/wave/fpc/fpc.go.

Wave is the threshold voting engine. Each round:
1. Sample k peers via Prism/Cut (wave.go:106)
2. Collect votes with timeout (wave.go:115-131)
3. Compare yes votes against alpha threshold (wave.go:154)
4. If threshold met and matches current preference, increment confidence (
   wave.go:159)
5. If confidence reaches beta, finalize (wave.go:161-163)
6. If threshold met but contradicts preference, flip and reset confidence (
   wave.go:166-168)

FPC variant:  $\alpha = \text{ceil}(\text{theta} * k)$  where theta is PRF-selected per phase.
theta in  $[\text{theta\_min}, \text{theta\_max}]$ , selected via SHA-256(seed || phase).
(fpc.go:36-38)

Properties proved:
- Monotonic confidence: once confidence reaches c, it can only increase or
  reset to 0
- Finalization stability: once decided, never undecided (wave.go:98-102)
- FPC range: alpha is always in  $[\text{ceil}(\text{theta\_min} * k), \text{ceil}(\text{theta\_max} * k)]$ 
- Convergence: under honest majority, confidence reaches beta in  $O(\text{beta})$ 
  rounds
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Tactic

namespace Protocol.Wave

/-- Wave configuration (models wave.Config) -/
structure WaveConfig where
  k      : Nat      -- sample size
  alpha  : Nat      -- threshold (fixed mode)
  beta   : Nat      -- confidence target
  hk     : 0 < k
  ha     : alpha < k
  hb     : 0 < beta

/-- FPC configuration (models fpc.Selector) -/
structure FPConfig where
  thetaMin : Rat      -- minimum theta
  thetaMax : Rat      -- maximum theta
  htmin    : 0 < thetaMin
  htmax    : thetaMin < thetaMax
  htbound  : thetaMax < 1

/-- Wave item state (models wave.WaveState) -/
structure WaveState where
```

```

    decided      : Bool
    preference    : Bool      -- true = commit, false = skip
    confidence    : Nat       -- consecutive confirming rounds

/-- A single round outcome -/
structure RoundResult where
  yesVotes      : Nat
  totalVotes    : Nat

/-- Step function: one round of Wave (models wave.Tick) -/
def step (cfg : WaveConfig) (s : WaveState) (r : RoundResult) : WaveState :=
  if s.decided then s -- wave.go:98-102: early return if decided
  else
    let thresholdMet := r.yesVotes    cfg.alpha
    if thresholdMet then
      if s.preference then
        -- Consecutive confirmation (wave.go:157-163)
        let newConf := s.confidence + 1
        if newConf    cfg.beta then
          { decided := true, preference := true, confidence := newConf }
        else
          { decided := false, preference := true, confidence := newConf }
      else
        -- Preference flip (wave.go:166-168)
        { decided := false, preference := true, confidence := 1 }
    else
      let noThresholdMet := (r.totalVotes - r.yesVotes)    cfg.alpha
      if noThresholdMet then
        if ¬s.preference then
          let newConf := s.confidence + 1
          if newConf    cfg.beta then
            { decided := true, preference := false, confidence := newConf }
          else
            { decided := false, preference := false, confidence := newConf }
        else
          { decided := false, preference := false, confidence := 1 }
      else
        -- No threshold met, confidence resets (wave.go:170)
        { decided := false, preference := s.preference, confidence := 0 }

/-- Once decided, step returns the same state -/
theorem decided_is_stable (cfg : WaveConfig) (s : WaveState) (r : RoundResult)
  (h : s.decided = true) : step cfg s r = s := by
  simp [step, h]

/-- Confidence only increases when preference matches threshold -/
theorem confidence_monotone_on_match
  (cfg : WaveConfig) (s : WaveState) (r : RoundResult)
  (hnd : s.decided = false)
  (hpref : s.preference = true)
  (hthresh : r.yesVotes    cfg.alpha)
  : (step cfg s r).confidence = s.confidence + 1 := by
  simp [step, hnd, hpref, hthresh]
  split <;> simp

```

```

/-- After beta consecutive matching rounds, the item is decided -/
theorem finalization_after_beta
  (cfg : WaveConfig)
  (s : WaveState)
  (r : RoundResult)
  (hnd : s.decided = false)
  (hpref : s.preference = true)
  (hthresh : r.yesVotes < cfg.alpha)
  (hconf : s.confidence + 1 < cfg.beta)
  : (step cfg s r).decided = true := by
simp [step, hnd, hpref, hthresh]
split
· simp
· omega

end Protocol.Wave

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Protocol/Wave.lean>

White papers: <https://papers.lux.network>

Related white papers:

- [lux-consensus.tex](#)
- [lux-fpc-consensus.tex](#)